	Ensure proper cleanup using `clearTimeout` to avoid memory leaks. Include `value` and `delay` in the dependency array of the `useEffect` hook.

With clear requirements in place, listing 4.5, located in `hooks/use-debounce/useDebounce.ts`, presents the implementation of the custom hook, `useDebounce`.

Listing 4.5 Implementation of useDebounce hook

```
const useDebounce = <T>({ value, delay = 500 }: useDebounceProps<
  const [debouncedValue, setDebouncedValue] = useState<T>(value);
  useEffect(() => {
    const handler = setTimeout(() => setDebouncedValue(value), de
    // Cleanup timeout on value or delay change
    return () => clearTimeout(handler); #C
  }, [value, delay, setDebouncedValue]); #D
  return debouncedValue;
};
```

Listing 4.6, located in `use-cases/DebounceExample.tsx`, builds an input UI upon this custom hook, `useDebounce`.

Listing 4.6 Use case to use useDebounce

```
const DebounceExample: React.FC = () => {
  const [searchTerm, setSearchTerm] = useState<string>('');
  const debouncedSearchTerm = useDebounce<string>({
    value: searchTerm,
    delay: 500,
  });
  const handleSearch = (e: React.ChangeEvent<HTMLInputElement>) =
    setSearchTerm(e.target.value);
  useEffect(() => {
    if (debouncedSearchTerm) {
      console.log('Performing search for:', debouncedSearchTerm);
    }
  }, [debouncedSearchTerm]);
  return (
    <input
      type="text"
      value={searchTerm}
```